

Supplementary Material for Beyond Context Windows: Evaluating Structured LLM-Based Clustering of Large-Scale Multilingual Corpora

Davide Colla ¹, Elisa Di Nuovo¹, Nicolas Stefanovitch^{1*}, Bertrand De Longueville¹, and Charles MacMillan¹

European Commission, Joint Research Centre, Via Enrico Fermi 2749, Ispra (VA),
Italy
{davide.colla, elisa.di-nuovo, nicolas.stefanovitch,
bertrand.de-longueville, charles.macmillan}@ec.europa.eu

This document supplements the main paper. It reports the limitations (Sect. S1), computational cost (Sect. A.1), information on the human annotators (Sect. A.2), all prompts used by the agentic baseline (Sect. A.3) and by the structured pipeline (Sect. A.4), the embedding thresholds used for filtering (Sect. A.5), the BABILong evaluation used to set the chunk size (Sect. A.6) and detailed classification results (Sect. A.7).

S1 Limitations

Several limitations constrain the generalisability and performance of our approach:

Single dataset. Our tests are carried out on a single dataset. However, this dataset was chosen for three reasons: it collects real-world data with high policy relevance, there is prior work on clustering using it, and it is highly multilingual.

Short text bottleneck. Proposal titles average approximately 25 tokens, providing limited context for semantic discrimination. This particularly affects micro-category clustering, where subtle thematic differences cannot be reliably detected. Future work should investigate whether full proposal texts or abstracts improve fine-grained clustering quality.

Independence assumption. Processing chunks independently prevents the model from identifying semantic relationships that span chunk boundaries. Whilst overlap mitigates this partially, proposals discussing similar themes but appearing in different chunks may be assigned to semantically equivalent but nominally distinct clusters, requiring post-hoc merging.

* As of 1st February 2026, Nicolas Stefanovitch is no longer employed at the European Commission, Joint Research Centre.

Language-specific performance variation. Whilst our qualitative evaluation included three languages (English, Italian, French), the multilingual performance may vary across the full languages represented in CoFE, which go beyond the official 24 EU languages. Embedding models may exhibit different quality-coverage trade-offs for lower-resourced languages, and LLM clustering performance likely depends on language-specific pre-training exposure.

Ad hoc pipeline decisions. Prompts are tailored for the case study and different prompts can lead to different outcomes. The pipeline requires specifying target cluster counts at the merging stage (10 for macro-categories). Whilst this aligns with the CoFE ground truth structure, determining appropriate cluster numbers for unlabelled corpora remains an open challenge.

Computational cost. Each chunk requires separate LLM inference for local clustering, followed by additional inference for merging and classification. Computational costs remain substantial compared to classical clustering approaches.

Ground truth limitations. We treat citizen-selected CoFE categories as ground truth, but proposals may belong to multiple categories or may be miscategorised by citizens. For the micro-categories, where no ground truth is available, the LLM-as-a-judge categories have been used. This creates different types of noise that has impact on V-measure and F1 scores.

A Appendix

A.1 Computational cost

All experiments were performed on a machine with an AMD EPYC 7282 16-core processor, 500 GB of RAM, and a Quadro 8000 GPU with 48 GB of memory. Each complete experiment took approximately 4 hours, of which 3.5 hours were dedicated to classification.

Table S1 compares the computational cost of our LLM-based pipeline with the graph-based approach from [?]. The graph-based method is approximately $22\times$ faster, primarily because it relies on BiLSTM-based LASER embeddings (computed in under one minute) and efficient Louvain community detection (a few seconds), with pairwise similarity computation as the dominant step (~ 10 minutes with parallelisation). In contrast, our LLM-based pipeline requires multiple inference passes for local clustering, merging, and classification, with the zero-shot classification step alone accounting for 3.5 hours. This substantial computational overhead is a practical trade-off for the interpretability and semantic abstraction capabilities that LLMs provide.

A.2 Annotators Information

The annotation has been carried out by two annotators selected among the authors: one female Italian native speaker, with C2 English, B2 French and a male French native speaker, with C2 English and C2 Italian.

Table S1: Computational cost comparison between approaches.

Approach	Hardware	Runtime
LLM Pipeline (ours)	16-core CPU, Quadro 8000 GPU	~4 hours
Graph-based (SC) [?]	32-core CPU, RTX 8000 GPU	~10–12 min

A.3 Agent Prompts

Planner node

You are a Task Planner for a specialized text clustering agent, whose goal is to cluster a very large collection of texts. The tasks you generate must be strictly guided by the usage rules outlined below.

1. Examine the user’s query and determine the most appropriate sequence of tasks to be executed. For each task, list the tools to be called in order to accomplish it.
2. Construct a plan by separating logical operations into tasks, and identify the tools as the steps required to complete each task.
3. Provide short and meaningful names for tasks and tools.
4. In `(*@texttt{tool\_calls}@*)`, list the tools in the order in which they should be used, observing the constraints mentioned above.
5. Do not call the same tool multiple times within the same task. Provide as output a JSON with the following structure, and do not output anything else.

Before producing the output, check that the JSON is valid. Correct it if it is not valid.

```
class ToolCall(BaseModel):
    tool_name: Annotated[str, ""]
    goal_of_the_tool_call: Annotated[str, "The information you want to obtain from the tool call."]
class PlannedTask(BaseModel):
    task_title: Annotated[str, "The title of the task."]
    task_description: Annotated[str, "A description of the task to be executed."]
    tool_calls: Annotated[List[ToolCall], "The list of tools to be called to accomplish the task. The list of tools must be ordered according to the sequence in which they should be used."]
Output:
class Plan(BaseModel):
    plan: Annotated[List[PlannedTask], "The list of tasks required to accomplish the objective. The list of tasks must be ordered according to the sequence in which they should be executed."]
User question:
{user_message}
```

Fig.S1: The instruction provided to the planner node in the agent pipeline.

Step writer node

You are a Python code writer agent. Your goal is to write a Python function to accomplish the task provided by the user.

The code generation must observe the following points:

1. The code you generate must be a valid and complete Python function.
2. Write a single function that includes all the code needed to accomplish the task.
3. Include all the imports required to run the function.
4. Comment the code to make it easy to understand.
5. Your complete response will be stored in a Python file and executed.
6. If your task requires translation or any other service that requires a Large Language Model, you may use the endpoint {custom LLM endpoint URL}, which is compatible with the OpenAI SDK. To run a query, you must set the base_url parameter to the endpoint URL. You may use the model qwen3-coder-30b-a3b-instruct, and you can assume that the API key is stored in an environment variable called (*@\texttt{API_KEY}@*).

Example code to connect the OpenAI client:

```
client = OpenAI(
    api_key='YOUR TOKEN HERE',
    base_url="{custom LLM endpoint URL}")
chat_completion = client.chat.completions.create(
    model="qwen3-coder-30b-a3b-instruct",
    messages=[...],
    stream=False,
)
```

7. Please provide helpful messages so that the code is easy to debug.
8. If you plan long-running operations, provide progress monitoring using tqdm.

When writing the code, note that it will be executed using a sandbox tool. Please provide output messages that can be used to inform the user. The code will be executed through a Pyodide sandbox; therefore, use only compatible packages.

You will be provided with the definition of the plan you are working on, as well as the specific task within the plan for which you need to write the code.

Provide as output only the Python code, and do not output anything else.

This is the user question:

```
{user_question}
```

You are in the process of completing the following task:

```
{task_description}
```

In particular, you are generating the following function:

```
{step_description}
```

Fig. S2: The instruction provided to the step writer node in the agent pipeline.

Main writer node

You are a Python code writer agent. Your goal is to write a single Python script that combines different functions previously built. The code generation must observe the following points:

1. The code you generate must be a valid and complete Python script.
2. Include all the imports required to run the script.
3. Comment the code to make it easy to understand.
4. Your complete response will be stored in a Python file and executed. Ensure that you provide only valid Python code in the output.
5. When writing the code, note that it will be executed using a sandbox tool. Please provide output messages that can be used to inform the user. The code will be executed through a Pyodide sandbox. You will be provided with a complete plan to achieve a clustering task. Each task within the plan consists of tools, which are Python functions described with their code.

Provide as output only the Python code, and do not output anything else.

Here is the plan with the corresponding Python functions:
{plan_with_description}

Fig. S3: The instruction provided to the main writer node in the agent pipeline.

Main fixer node

You are a Python code writer agent. Your goal is to write a single Python script that fixes and combines previously built functions. The code generation must observe the following points:

1. The code you generate must be a valid and complete Python script.
2. Include all the imports required to run the script.
3. Comment the code to make it easy to understand.
4. Your complete response will be stored in a Python file and executed. Ensure that you provide only valid Python code in the output.
5. When writing the code, note that it will be executed using a sandbox tool. Please provide output messages that can be used to inform the user. The code will be executed through a Pyodide sandbox. You will be provided with a complete plan to achieve a clustering task. Each task within the plan consists of tools, which are Python functions described with their code.

Provide as output only the Python code, and do not output anything else.

Here is the executed Python code:
{code}

Here is the standard output of the execution:
{std_out}

Here is the generated error:

```
{std_err}
```

Fig. S4: The instruction provided to the main fixer node in the agent pipeline.

Generating user answer node

You are an assistant designed to answer user questions.
 A user has requested the completion of a task. The task has been accomplished through code that was generated by an LLM and executed locally.
 You will receive the executed code along with its output. Provide a response to the user.
 Here is the user question:
 {user question}
 Here is the executed code:
 {code}
 Here is the output:
 {output}

Fig. S5: The instruction provided to the user answer node in the agent pipeline.

User question

I have 8,000 texts originating from European consultations.
 I would like to cluster them according to the topics they address.
 There should be 10 clusters.
 The texts may be written in any of the 27 languages of the European Union member states.
 The language of each document is known and validated; do not validate the language again.
 Each entry in the dataset follows a JSON schema. The language of each document is encoded in an input field.
 The dataset is stored in a JSON file located at {path}.
 The JSON schema of each entry is as follows:
 {"text": "The text of the document",
 "id": "The ID of the document",
 "language": "The two-letter language code of the document (e.g., en, fr, de, etc.)"}
 If you need to translate the documents into English, the translations are already available at {path}.
 For each entry, if the language is not en, an additional field called text_en is provided, which contains the English translation of the original text.
 As output, I want a JSON file containing a list of objects, one for each input text.

```
Each JSON object must have the following structure:
{"id": "The ID of the document",
"cluster": "The name of the cluster"}
Additionally, please provide an evaluation of the clustering using the
silhouette metric in a dedicated file called results.txt.
I want both the silhouette score for each sample and the average
silhouette score.
All output files should be saved in the current directory.
```

Fig. S6: The instruction provided to the agent pipeline.

A.4 LLM-Based Clustering Prompts

Chunk cluster extraction

```
The task is to cluster a list of articles.
In order to design the clusters, please consider the most relevant and
emerging topics mentioned within articles.
Provide in output a JSON with the following structure, do not output
anything else.
Before outputting check the JSON is valid. Correct it if it is not
class Cluster(BaseModel):
name: str # a short name describing the cluster in English
summary: str # redact an executive summary of the information of all
the title that belong to that cluster
Output:
List[Cluster] # the list of cluster definition
```

Fig. S7: The instruction provided to the LLM for the proposals clustering task.

Clusters merging

```
You are provided with a list of cluster names together with their
descriptions.
Your task is to merge these lists, by merging very similar cluster
together.
Propose a final list of 10 clusters. Do not propose code, propose
directly the list.
Output a JSON with the following structure, do not output anything
else.
Before outputting check the JSON is valid. Correct it if it is not.
class Cluster(BaseModel):
name: str # a short name describing the cluster in English
summary: str # redact an executive summary of the information of all
the title that belong to that cluster
```

```

number: int # the ordinal number of the cluster, from 1 to 10
Output:
List[Cluster] # the list of cluster definition

```

Fig. S8: The instruction provided to the LLM for the clusters merging task.

Clusters mapping

Your task is to find the appropriate cluster name of a given list of elements. To find the best cluster, consider the semantically most similar one. The cluster names are the following 10:
{CoFE categories with description}

Once you find the most semantically similar cluster for the list, output a JSON with the following structure, do not output anything else.

Before providing an output check the JSON is valid. Correct it if it is not.

```

class ClusterMap(BaseModel):
    origianl_cluster_name: str # The original name of the cluster
    mapped_cluster: str # The name of the corresponding cluster
Output:
List[ClusterMap] # the list of cluster mappings

```

Fig. S9: The instruction provided to the LLM for the clusters mapping task.

Proposal classification

Your task is to find the most appropriate class for a given text. In what follows you can find the set of classes together with their definitions.

You can choose one of the following classes:

- Climate Change and Sustainability: The European Union must prioritize climate change mitigation and adaptation, investing in renewable energy, reducing carbon emissions, and promoting sustainable practices across all sectors.
- European Integration and Democracy: The EU should strengthen its democratic institutions, increase citizen participation, and promote a sense of European identity to build a more united and democratic Europe.
- Social Justice and Equality: The EU should prioritize social justice, promoting equality, human rights, and the well-being of all its citizens, and addressing issues such as poverty, discrimination, and inequality.
- Economic Growth and Competitiveness: The EU should focus on promoting economic growth, innovation, and competitiveness, while

ensuring that the benefits of growth are shared fairly among all member states and citizens.

- Migration and Integration: The EU should develop a comprehensive and humane migration policy, ensuring the safe and dignified treatment of asylum seekers and refugees, while promoting integration and social cohesion.
- Health and Wellbeing: The EU should prioritize the health and wellbeing of its citizens, investing in high-quality healthcare systems, promoting mental health, and addressing the social determinants of health.
- Education and Skills: The EU should promote high-quality education, vocational training, and lifelong learning opportunities, enabling citizens to acquire the skills needed to thrive in a rapidly changing labor market.
- Digitalization and Technology: The EU should harness the potential of digitalization and technology to drive innovation, improve productivity, and enhance the quality of life for its citizens, while addressing the challenges and risks associated with these developments.
- Security and Defense: The EU should prioritize the security and defense of its citizens, addressing the threats posed by terrorism, cybercrime, and other external challenges, while promoting cooperation and solidarity among its member states.
- Governance and Institutions: The EU should strengthen its governance and institutions, ensuring that they are transparent, accountable, and responsive to the needs and concerns of its citizens, while promoting democratic participation and engagement.

Output a JSON with the following structure, do not output anything else.

Before outputting check the JSON is valid. Correct it if it is not

Output:

```
class: str # the class corresponding to the given text.
```

Fig. S10: The instruction provided to the LLM for the clusters mapping task.

LLM as a judge on proposal

Your task is to decide whether a sentence belongs to a group, given the group title and its description.

Give a score from 1 to 5 to judge if the sentence belongs to the group, where 1 means that the sentence does not belong to the set, while 5 means that it definitely belongs to it.

Provide in output a JSON with the following structure, do not output anything else.

```
class Score(BaseModel):
    score: int # The score indicating if the sentence belongs to the group
                identified by title and description. The score can range from 1 to 5,
```

```

Where 1 means that the sentence is not belonging to the group, while
5 means that it definitely belongs to it.
Output:
Score # the score assigned to the sentence
Here is the group title:
{cluster name}
Here is the group description:
{cluster summary}
Here is the sentence:
{proposal title}

```

Fig. S11: The instruction provided to the LLM for the proposal scoring task.

A.5 Embeddings Threshold Scores

The thresholds used are reported in Table S2.

Table S2: Embedding models and cosine similarity thresholds for post-cluster filtering.

Emb. Model	Dimensionality	Threshold
Qwen	Full	0.70
Qwen_128	128	0.76
BERT	768	0.62
E5	768	0.85
LASER	1024	0.85

A.6 BABILong Evaluation

To tune the chunk size, we use BABILong results on the first five tasks to guide the selection of chunk sizes, adopting the context length corresponding to the best performance as an estimate of the model’s optimal operating regime.

BABILong is a benchmark designed to evaluate the long-context capabilities of LLMs under controlled needle-in-a-haystack conditions. It consists of several question-answering tasks that progressively stress different aspects of long-context reasoning, including simple retrieval, multi-hop reasoning, distractor robustness, and the ability to locate sparse but relevant information embedded within large volumes of irrelevant text.

The results obtained are reported in Fig. S12.

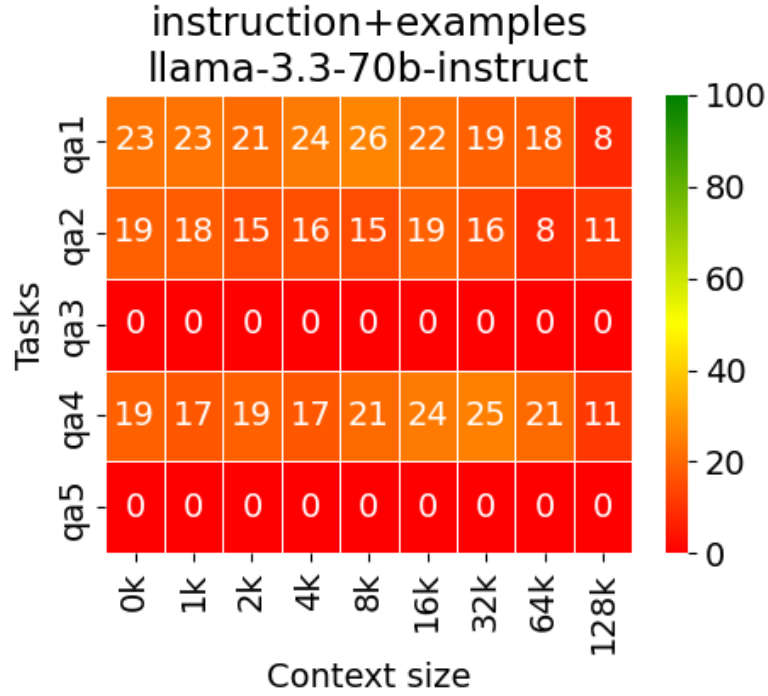


Fig. S12: QA1–QA5 BABILong results of the selected LLM, LLaMA 3.3 70 B.

The model tested, LLaMA 3.3 70B, underperforms compared to the models evaluated in the BABILong leaderboard.¹ In two out of five tasks it is not able to reply. In the other three tasks, however it maintains a quite constant performance which only deteriorates at 64k-128k. On average, the best performance is achieved with context length 16k.

A.7 Zero-Shot Classification: Detailed Results

¹ BABILong leaderboard (last accessed 13 December 2025): <https://huggingface.co/spaces/RMT-team/babilong>.

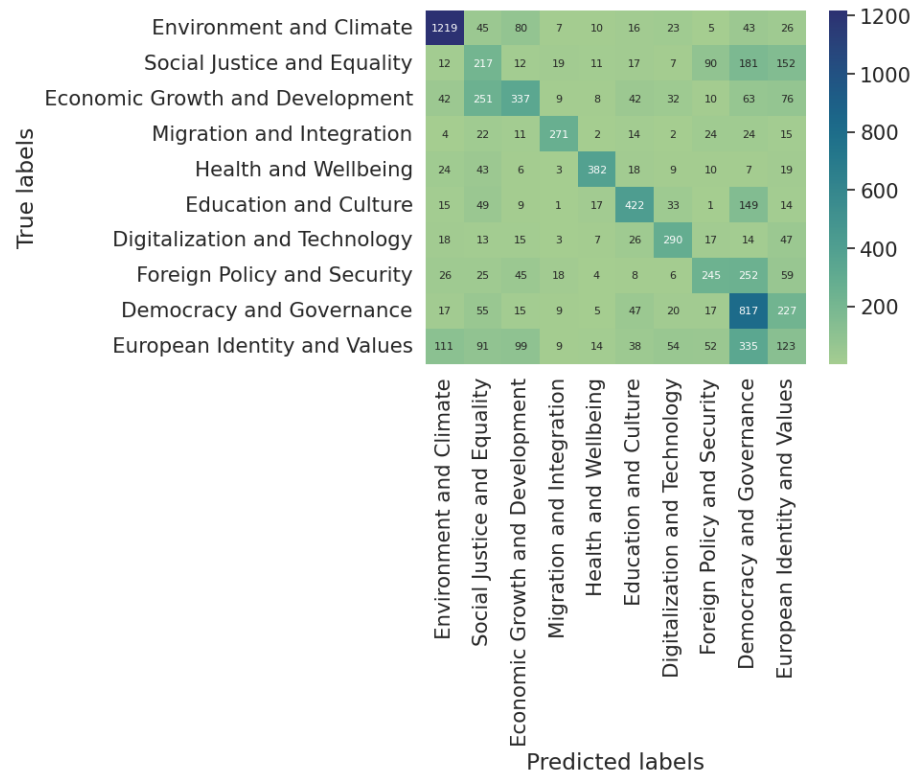


Fig. S13: Confusion matrix of the LLM pipeline with chunk size 16k and overlap.

Table S3: 16k - 10 clusters with overlap.

	precision	recall	f1-score	support
Environment and Climate	0.819	0.827	0.823	1475
Social Justice and Equality	0.268	0.302	0.284	719
Economic Growth and Development	0.536	0.387	0.450	873
Migration and Integration	0.777	0.697	0.734	390
Health and Wellbeing	0.830	0.733	0.779	522
Education and Culture	0.651	0.594	0.622	713
Digitalization and Technology	0.609	0.644	0.626	450
Foreign Policy and Security	0.520	0.356	0.423	694
Democracy and Governance	0.433	0.665	0.525	1233
European Identity and Values	0.162	0.133	0.146	933
accuracy	0.542	0.542	0.542	
macro avg	0.561	0.534	0.541	8002
weighted avg	0.549	0.542	0.538	8002

Table S4: 16k - 100 clusters with overlap.

	precision	recall	f1-score	support
Environment and Climate	0.750	0.790	0.770	1475
Social Justice and Equality	0.266	0.583	0.365	719
Economic Growth and Development	0.347	0.413	0.377	873
Migration and Integration	0.687	0.721	0.704	390
Health and Wellbeing	0.684	0.758	0.719	522
Education and Culture	0.513	0.729	0.602	713
Digitalization and Technology	0.657	0.698	0.677	450
Foreign Policy and Security	0.538	0.647	0.587	694
Democracy and Governance	0.754	0.292	0.421	1233
European Identity and Values	0.	0.	0.	933
accuracy	0.536	0.536	0.536	
macro avg	0.520	0.563	0.522	8002
weighted avg	0.524	0.536	0.506	8002

Table S5: 16k - 10 clusters without overlap.

	precision	recall	f1-score	support
Environment and Climate	0.799	0.834	0.816	1475
Foreign Policy and Security	0.228	0.114	0.152	719
Social Justice and Equality	0.308	0.298	0.303	873
Migration and Integration	0.765	0.700	0.731	390
Health and Wellbeing	0.835	0.729	0.779	522
Education and Culture	0.588	0.718	0.646	713
Digitalization and Technology	0.627	0.705	0.664	450
Economic Growth and Development	0.041	0.039	0.040	694
Democracy and Governance	0.469	0.795	0.590	1233
European Identity and Values	0.083	0.028	0.042	933
accuracy	0.512	0.512	0.512	
macro avg	0.474	0.496	0.476	8002
weighted avg	0.467	0.512	0.478	8002

Table S6: 20k - 10 clusters with overlap.

	precision	recall	f1-score	support
Environment and Climate	0.743	0.851	0.775	1475
Social Justice and Equality	0.269	0.309	0.269	719
Economic Growth and Development	0.521	0.442	0.455	873
Migration and Integration	0.786	0.659	0.693	390
Health and Wellbeing	0.805	0.741	0.750	522
Education and Culture	0.598	0.659	0.608	713
Digitalization and Technology	0.612	0.696	0.633	450
Foreign Policy and Security	0.466	0.316	0.349	694
Democracy and Governance	0.401	0.776	0.522	1233
European Identity and Values	0.201	0.104	0.105	933
accuracy	0.530	0.570	0.530	
macro avg	0.540	0.555	0.516	8002
weighted avg	0.522	0.570	0.512	8002